SENIOR DEVELOPER EDITION · 2026



JAVA 8 → 21

The Complete Interview Q&A Guide

60+ Questions · Code · Scenarios · Production Tips

TechTribers

26,200+ developers leveled up with this guide

WHAT'S INSIDE

Table of Contents

8 **Java 8 — The Functional Revolution** — Lambdas, Streams, Optional, java.time, CompletableFuture — 15 Qs

9 **Java 9 — Modules & API Polish** — JPMS, factory methods, Stream additions, Flow API — 8 Qs

11 **Java 10–11 (LTS) — Productivity** — var, String methods, HttpClient, Files helpers — 8 Qs

15 **Java 12–15 — Modern Syntax** — Switch expressions, text blocks, records, sealed (preview) — 10 Qs

17 **Java 16–17 (LTS) — Standardized** — Records, pattern matching, sealed classes, Stream.toList — 8 Qs

21 **Java 18–21 (LTS) — Project Loom Era** — Virtual threads, record patterns, sequenced collections — 12 Qs



RELEASED MARCH 2014 · LTS

Java 8

The Functional Revolution

The most impactful release ever: lambdas, the Stream API, `Optional`, method references, default methods, a redesigned date/time API, and `CompletableFuture` turned Java into a hybrid OO + functional language.

Q1 What is a lambda expression and what problem does it solve?

Beginner | Java 8

💡 THEORY

A lambda is a concise, anonymous implementation of a **functional interface** (an interface with exactly one abstract method, the SAM). Before Java 8 you needed verbose anonymous classes to pass behavior; lambdas let you treat behavior as data and pass it to methods, enabling a functional style.

⌨️ CODE

Lambda.java

```
// Old way — anonymous class
Runnable r1 = new Runnable() {
  public void run() { System.out.println("hi"); }
};
// Lambda
Runnable r2 = () -> System.out.println("hi");
Comparator<String> byLen = (a, b) -> a.length() - b.length();
```

▶️ OUTPUT

Both `r1` and `r2` print `hi`. The lambda compiles to an `invokedynamic` call site — not an extra `.class` file like anonymous classes.

⚠️ INTERVIEW TIP

A lambda is **not** just syntactic sugar for an anonymous class. `this` inside a lambda refers to the *enclosing* instance, whereas in an anonymous class it refers to the anonymous class itself.

Q2 What are functional interfaces? Name the core ones in `java.util.function`.

Beginner | Java 8

💡 THEORY

A functional interface has a single abstract method and can be the target of a lambda. `@FunctionalInterface` enforces this at compile time. The four core families are: `Function<T,R>` (transform), `Predicate<T>` (test→boolean), `Consumer<T>` (side-effect), and `Supplier<T>` (produce).

⌨️ CODE

FunctionalInterfaces.java

```
Function<Integer,Integer> sq = n → n * n;
Predicate<String> empty = String::isEmpty;
Consumer<String> log = System.out::println;
Supplier<Double> rnd = Math::random;
BiFunction<Integer,Integer,Integer> add = Integer::sum;
```

▶️ OUTPUT

`sq.apply(5)` → **25**, `empty.test("")` → **true**, `add.apply(2,3)` → **5**.

⚠️ INTERVIEW TIP

Know the primitive specializations (`IntFunction` , `ToIntFunction` , `IntPredicate` …). They exist to avoid autoboxing overhead in hot loops.

Q3 Explain the Stream API. How is it different from a Collection?

Intermediate | Java 8

💡 THEORY

A `Stream` is a lazy, single-use pipeline of operations over a source. A Collection *stores* elements; a Stream *computes* them on demand. Streams have **intermediate** ops (lazy: `map`, `filter`) and a **terminal** op (eager: `collect`, `forEach`) that triggers execution. Nothing runs until the terminal op.

⌨️ CODE

Streams.java

```
List<String> names = List.of("ann","bob","amy","al");
List<String> result = names.stream()
    .filter(n → n.startsWith("a"))
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
```

▶️ OUTPUT

`[AL, AMY, ANN]` — `bob` filtered out, rest upper-cased and sorted.

🎯 SCENARIO

Great for declarative data transformation (DTO mapping, report aggregation). The interviewer often asks: *"Can you reuse a stream?"* — No. A stream throws `IllegalStateException` once a terminal op has run.

Q4 map vs flatMap — when do you reach for flatMap?

🟡 Intermediate | Java 8

💡 THEORY

`map` is 1→1: it transforms each element. `flatMap` is 1→many then flattens: each element becomes a stream and all are concatenated into one. Use `flatMap` to flatten nested structures (list of lists) or to combine streams.

⌨️ CODE

FlatMap.java

```
List<List<Integer>> nested =
    List.of(List.of(1,2), List.of(3,4));
List<Integer> flat = nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

▶️ OUTPUT

`[1, 2, 3, 4]` — the two inner lists are flattened into a single stream.

⚠️ INTERVIEW TIP

If you ever write `.map(...).collect(...)` and get a `Stream<Stream<X>>` or `List<List<X>>` you didn't want — that's the signal you needed `flatMap`.

Q5 How does reduce() work? Explain identity, accumulator, combiner.

Advanced | Java 8

💡 THEORY

`reduce` folds a stream into a single value. The 3-arg form `reduce(identity, accumulator, combiner)`: **identity** is the start value (and result for an empty stream), **accumulator** combines partial result + element, and **combiner** merges partial results from parallel sub-streams. The combiner is only used in parallel execution.

⌨️ CODE

Reduce.java

```
int sum = Stream.of(1,2,3,4)
    .reduce(0, Integer::sum);

int len = Stream.of("a","bb","ccc")
    .reduce(0,
        (acc, s) → acc + s.length(),    // accumulator
        Integer::sum);                  // combiner
```

▶️ OUTPUT

`sum = 10`, `len = 6`. With a wrong combiner, parallel results become non-deterministic.

⚠️ INTERVIEW TIP

The identity must be a true identity: `combiner(identity, x) == x`. Picking `1` as identity for a sum silently corrupts parallel results.

Q6 What is Optional and how should it be used correctly?

🟡 Intermediate | Java 8

💡 THEORY

`Optional<T>` is a container that may or may not hold a non-null value, designed to model "no result" explicitly and reduce `NullPointerException`. Prefer `map` / `filter` / `orElseGet` over `isPresent()` + `get()`.

⌨️ CODE

Optionals.java

```
String city = Optional.ofNullable(user)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElse("UNKNOWN");
```

▶️ OUTPUT

Returns the city, or `"UNKNOWN"` if `user`, address, or city is null — no nested null checks.

⚠️ INTERVIEW TIP

Never use `Optional` as a field or method parameter — it's a **return type**. Use `orElseGet` (lazy) over `orElse` (eager) when the fallback is expensive.

Q7 Explain the four kinds of method references.

🟡 Intermediate | Java 8

💡 THEORY

A method reference is shorthand for a lambda that just calls one method. The four kinds: static (`Integer::parseInt`), bound instance (`obj::method`), unbound instance (`String::length`), and constructor (`ArrayList::new`).

⌨️ CODE

MethodRef.java

```
Function<String,Integer> p  = Integer::parseInt;     // static
Supplier<List<String>> s    = ArrayList::new;        // ctor
Function<String,Integer> ln = String::length;        // unbound
Consumer<String> print      = System.out::println;   // bound
```

▶️ OUTPUT

`p.apply("42")` →42, `ln.apply("hello")` →5. Equivalent to the lambda but cleaner.

⚠️ INTERVIEW TIP

The tricky one is **unbound**: `String::length` as a `Function<String,Integer>` — the first lambda parameter becomes the receiver.

Q8 Why were default methods added to interfaces?

🟡 Intermediate | Java 8

💡 THEORY

Default methods let interfaces ship a method body. The motivation was **backward compatibility**: adding `stream()` / `forEach()` to `Collection` without breaking the millions of existing implementations. Static interface methods were added for utility factories.

⌨️ CODE

DefaultMethod.java

```
interface Greeter {
  String name();
  default String greet() { return "Hi " + name(); }
  static Greeter of(String n) { return () → n; }
}
```

▶️ OUTPUT

`Greeter.of("Sam").greet()` → `"Hi Sam"`.

⚠️ INTERVIEW TIP

The **diamond problem**: if two interfaces give the same default method, the implementing class *must* override it and can call `InterfaceName.super.method()` to disambiguate.

Q9 Walk through the key Collectors. What does groupingBy do?

Advanced | Java 8

💡 THEORY

`Collectors` are reduction recipes for the `collect` terminal op. Common ones: `toList/toSet/toMap`, `joining`, `counting`, `summingInt`, `groupingBy` (build a `Map` of key→list), and `partitioningBy` (split by predicate into true/false buckets). `groupingBy` accepts a downstream collector for nested reduction.

⌨️ CODE

Collectors.java

```
Map<Dept,Long> headcount = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDept,
        Collectors.counting()));
```

▶️ OUTPUT

`{ENGINEERING=12, SALES=7, HR=3}` — a count per department via a downstream collector.

⚠️ INTERVIEW TIP

`Collectors.toMap` throws on duplicate keys unless you pass a merge function — a classic production bug.

Q10 Why is the old Date/Calendar API bad, and what does java.time fix?

Intermediate | Java 8

💡 THEORY

The legacy `java.util.Date` / `Calendar` are mutable, not thread-safe, 0-indexed months, and conflate date+time+zone. The `java.time` (JSR-310) API is immutable, thread-safe, and clearly separates concepts: `LocalDate`, `LocalTime`, `LocalDateTime`, `ZonedDateTime`, `Instant`, `Duration`, `Period`.

⌨️ CODE

JavaTime.java

```
LocalDate today = LocalDate.now();
LocalDate due   = today.plusDays(30);
Period gap      = Period.between(today, due);
boolean late    = due.isBefore(LocalDate.now());
```

▶️ OUTPUT

`gap.getDays()` → 30. All operations return **new** immutable instances — safe to share across threads.

⚠️ INTERVIEW TIP

`Instant` = a point on the UTC timeline (machine time). `LocalDateTime` has **no** zone. Mixing them up is the #1 date bug in interviews.

Q11 What is CompletableFuture and why is it better than Future?

Advanced | Java 8

💡 THEORY

`Future` only lets you block on `get()` — no composition. `CompletableFuture` is a non-blocking, composable promise: you can chain (`thenApply`), pipeline async tasks (`thenCompose`), combine two futures (`thenCombine`), and handle errors (`exceptionally` / `handle`).

⌨️ CODE

CompletableFuture.java

```
CompletableFuture
    .supplyAsync(() → fetchUser(id))
    .thenApply(User::getEmail)
    .thenAccept(System.out::println)
    .exceptionally(ex → { log(ex); return null; });
```

▶️ OUTPUT

Runs off the main thread, prints the email when ready, and routes any exception to the handler — no blocking `get()` .

🎯 SCENARIO

Aggregating several microservice calls in parallel: fire N `supplyAsync` calls, then `CompletableFuture.allOf(...)` to await all. Follow-up: *"Which thread pool runs it?"* — the common `ForkJoinPool` unless you pass an `Executor` .

Q12 When are parallel streams a good idea — and when a trap?

Advanced | Java 8

💡 THEORY

`parallelStream()` splits work across the common ForkJoinPool. It helps only with large datasets, CPU-bound, stateless, easily-splittable work. It hurts with small data, I/O-bound tasks, or ordered/stateful operations due to coordination overhead.

⌨️ CODE

Parallel.java

```
long count = bigList.parallelStream()
    .filter(Order::isValid)
    .mapToLong(Order::getAmount)
    .sum();
```

▶️ OUTPUT

Correct sum, computed across multiple cores — but only faster when the list is large and the work per element is non-trivial.

⚠️ INTERVIEW TIP

All parallel streams share the **same** common pool. One blocking I/O task can starve the whole JVM. Never do blocking calls in a parallel stream — that's what virtual threads (Java 21) are for.

Q13 Difference between findFirst() and findAny()?

🟡 Intermediate | Java 8

💡 THEORY

Both are short-circuiting terminal ops returning an `Optional`. `findFirst` respects encounter order and returns the first element. `findAny` may return any element — giving the runtime freedom to pick whatever a parallel worker finds first, so it can be faster in parallel.

⌨️ CODE

FindFirstAny.java

```
Optional<Integer> a = list.stream()
    .filter(n → n > 10).findFirst();
Optional<Integer> b = list.parallelStream()
    .filter(n → n > 10).findAny();
```

▶️ OUTPUT

`findFirst` deterministically returns the first >10; `findAny` on a parallel stream may return any element >10.

⚠️ INTERVIEW TIP

In a *sequential* stream, `findAny` usually behaves like `findFirst` — but never rely on that; it's not guaranteed.

Q14 What does Stream.collect with joining/peek do? Is peek safe?

Intermediate | Java 8

💡 THEORY

`Collectors.joining(delim, prefix, suffix)` concatenates strings. `peek` is an intermediate op for observing elements (debugging/logging) without consuming the stream — but because it's lazy, it only runs if a terminal op pulls elements, and JIT may skip it under certain optimizations.

⌨️ CODE

JoiningPeek.java

```
String csv = Stream.of("a","b","c")
    .peek(x → log.debug("saw {}", x))
    .collect(Collectors.joining(", ", "[", "]"));
```

▶️ OUTPUT

`[a, b, c]`, with each element logged as it flows through.

⚠️ INTERVIEW TIP

Never use `peek` for business logic / mutation. It's strictly for diagnostics — the spec allows the implementation to elide it.

Q15 Explain lazy evaluation & short-circuiting in streams.

Advanced | Java 8

💡 THEORY

Intermediate ops are **lazy** — they build a pipeline but don't execute until a terminal op runs. Elements are then pulled one at a time through the whole chain (not stage-by-stage over the whole collection). Short-circuiting ops (`limit` , `anyMatch` , `findFirst`) can stop early, so infinite streams via `iterate` / `generate` are usable.

⌨️ CODE

Lazy.java

```
List<Integer> first5Even = Stream
    .iterate(1, n → n + 1)    // infinite
    .filter(n → n % 2 == 0)
    .limit(5)                  // short-circuits
    .collect(Collectors.toList());
```

▶️ OUTPUT

`[2, 4, 6, 8, 10]` — terminates despite an infinite source because `limit(5)` short-circuits.

🎯 SCENARIO

Interviewers love: *"How many times does filter run here?"* — exactly until 5 even numbers are found (i.e. through 10), not over an infinite set, proving per-element pull semantics.



RELEASED SEPTEMBER 2017

Java 9

Modules & API Polish

The Java Platform Module System (JPMS) reshaped how large apps are structured, while immutable collection factories, new `Stream` operations, private interface methods, JShell, and the reactive `Flow` API rounded out the release.

Q16 What is the Java Platform Module System (JPMS)?

Advanced | Java 9

💡 THEORY

JPMS (Project Jigsaw) adds a module layer above packages. A `module-info.java` declares what a module `requires` and which packages it `exports`. This gives strong encapsulation (non-exported packages are truly hidden, even via reflection) and reliable configuration (missing dependencies fail at startup, not runtime).

⌨️ CODE

module-info.java

```
module com.shop.orders {
  requires com.shop.payments;
  exports com.shop.orders.api;
  // internal pkgs stay hidden
}
```

▶️ OUTPUT

Only `com.shop.orders.api` is visible to other modules; internal packages cannot be imported or reflected into.

⚠️ INTERVIEW TIP

Code with no `module-info` runs on the **classpath** in the "unnamed module," which can read everything — that's why most apps still don't fully modularize. Know `opens` (reflection) vs `exports` (compile access).

Q17 What do List.of / Set.of / Map.of give you?

Beginner | Java 9

💡 THEORY

Static factory methods that create compact, **immutable** collections in one line. They reject `null` elements, throw on duplicate keys, and return space-efficient implementations. They replace the verbose `Collections.unmodifiableList(Arrays.asList(...))` idiom.

⌨️ CODE

Factories.java

```
List<String> l = List.of("a", "b", "c");
Map<String,Integer> m = Map.of("x", 1, "y", 2);
l.add("d"); // throws UnsupportedOperationException
```

▶️ OUTPUT

Any mutation throws `UnsupportedOperationException` — these collections are truly immutable, not just unmodifiable views.

⚠️ INTERVIEW TIP

`List.of()` rejects `null` (NPE), whereas `Arrays.asList()` allows nulls and is fixed-size but element-mutable. Different contracts.

Q18 Explain takeWhile, dropWhile and the new iterate.

🟡 Intermediate | Java 9

💡 THEORY

`takeWhile(p)` takes the leading prefix while the predicate is true, then stops. `dropWhile(p)` discards the leading prefix and keeps the rest. Java 9 also added a 3-arg `Stream.iterate(seed, hasNext, next)` — a functional for-loop with a built-in termination condition.

⌨️ CODE

TakeWhile.java

```
Stream.of(2,4,6,7,8).takeWhile(n → n%2==0);
Stream.of(2,4,6,7,8).dropWhile(n → n%2==0);
Stream.iterate(1, n → n ⩽ 100, n → n * 2);
```

▶️ OUTPUT

`takeWhile` → `[2,4,6]` ; `dropWhile` → `[7,8]` ; `iterate` → `1,2,4,...,64` .

⚠️ INTERVIEW TIP

They operate on the **prefix**, not a global filter. `takeWhile` stops at the *first* false — later matching elements are NOT included (unlike `filter`).

Q19 What are private interface methods for?

Intermediate | Java 9

💡 THEORY

Java 9 allows `private` (and `private static`) methods in interfaces. They let multiple default methods share common code without exposing that helper as part of the public API — fixing a gap left by Java 8 default methods.

⌨️ CODE

PrivateIface.java

```
interface Audit {
  default void created() { log("CREATE"); }
  default void deleted() { log("DELETE"); }
  private void log(String op) {
    System.out.println(op + " @ " + Instant.now());
  }
}
```

▶️ OUTPUT

Both defaults reuse `log()`, which stays hidden from implementers and callers.

⚠️ INTERVIEW TIP

Private interface methods can't be `abstract` and can't be overridden — they're an implementation detail of the interface itself.

Q20 What new Optional methods arrived in Java 9?

🟡 Intermediate | Java 9

💡 THEORY

Three additions: `ifPresentOrElse(action, emptyAction)` handles both branches; `or(supplier)` returns an alternative `Optional` when empty; and `stream()` turns an `Optional` into a 0-or-1 element stream — perfect for flat-mapping away empties.

⌨️ CODE

Optional9.java

```
opt.ifPresentOrElse(
    v → System.out.println("got " + v),
    () → System.out.println("empty"));

List<User> users = ids.stream()
    .map(repo::findById)    // Optional<User>
    .flatMap(Optional::stream)
    .toList();
```

▶️ OUTPUT

`Optional::stream` elegantly drops all empty results, leaving only present users.

🎯 SCENARIO

Replacing the old `.filter(Optional::isPresent).map(Optional::get)` chain with a single `flatMap(Optional::stream)` — a common "clean this up" interview ask.

Q21 What is JShell and why does it matter?

Beginner | Java 9

💡 THEORY

JShell is Java's official REPL (Read-Eval-Print Loop). You can type expressions and statements and get immediate feedback without writing a class or `main` method — ideal for learning, prototyping APIs, and quick experiments.

⌨️ CODE

```jshell
jshell> var x = IntStream.rangeClosed(1,5).sum()
x ⟹ 15
jshell> "hi".repeat(3)
$2 ⟹ "hihihi"
```

▶️ OUTPUT

Each line evaluates instantly and binds an auto-named variable — no boilerplate, no compile step.

⚠️ INTERVIEW TIP

JShell is a tool, not a language change — but mentioning it signals you actually experiment with new APIs hands-on.

Q22 What is the Flow API (Reactive Streams)?

Advanced | Java 9

💡 THEORY

`java.util.concurrent.Flow` standardizes the Reactive Streams contract with four interfaces: `Publisher`, `Subscriber`, `Subscription`, and `Processor`. Its key idea is **back-pressure**: the subscriber calls `subscription.request(n)` to pull only as many items as it can handle, preventing fast producers from overwhelming slow consumers.

⌨️ CODE

Flow.java

```
class Sub implements Flow.Subscriber<String> {
  public void onSubscribe(Flow.Subscription s) {
    s.request(1);        // back-pressure: ask for 1
  }
  public void onNext(String item) { /* handle */ }
  public void onError(Throwable t) {}
  public void onComplete() {}
}
```

▶️ OUTPUT

The subscriber controls flow rate — it receives one item, processes it, then requests the next.

⚠️ INTERVIEW TIP

The JDK only ships the **interfaces** (plus `SubmissionPublisher`). Real reactive work uses RxJava / Project Reactor, which implement this same contract.

Q23 What is Stream.ofNullable and where is it useful?

Beginner | Java 9

💡 THEORY

`Stream.ofNullable(x)` returns an empty stream if `x` is null, otherwise a single-element stream. It removes null guards inside `flatMap` chains where a lookup might return null.

⌨️ CODE

OfNullable.java

```
List<String> vals = keys.stream()
    .flatMap(k → Stream.ofNullable(map.get(k)))
    .toList();
```

▶️ OUTPUT

Missing keys (null values) simply contribute nothing — no `NullPointerException`, no explicit filtering.

⚠️ INTERVIEW TIP

Contrast with `Stream.of(x)`, which would create a stream containing `null` and blow up downstream.



JAVA 10 (MAR 2018) · JAVA 11 LTS (SEP 2018)

Java 10–11

Developer Productivity & the Second LTS

Java 10 introduced `var` local type inference; Java 11 — the LTS most enterprises jumped to from 8 — added the standard `HttpClient`, rich new `String` methods, `Files.readString/writeString`, and the ability to run a single source file directly.

Q24 What is var? Is Java now dynamically typed?

Intermediate | Java 10

💡 THEORY

`var` is **local variable type inference**. The compiler infers the static type from the initializer at compile time — Java stays strongly, statically typed. `var` only works for local variables with an initializer (not fields, method params, or return types).

⌨️ CODE

Var.java

```
var list = new ArrayList<String>();   // ArrayList<String>
var count = 10;                       // int
for (var e : list) { /* String */ }
// var x;         // ERROR: no initializer
```

▶️ OUTPUT

Identical bytecode to writing the explicit type. Reassigning `count = "hi"` still fails to compile — the type is locked to `int`.

⚠️ INTERVIEW TIP

Don't overuse it. `var result = process();` hides the type and hurts readability. Use it when the RHS already makes the type obvious.

Q25 Why allow var in lambda parameters (Java 11)?

🟡 Intermediate | Java 11

💡 THEORY

Java 11 permits `var` in lambda parameter lists. The sole benefit is being able to attach annotations (like `@NonNull`) to an otherwise-inferred parameter, while keeping a uniform syntax. You must use `var` for *all* params or none.

⌨️ CODE

VarLambda.java

```
BiFunction<Integer,Integer,Integer> add =
    (var a, var b) -> a + b;

list.forEach((@NonNull var s) -> process(s));
```

▶️ OUTPUT

Compiles identically to `(a, b) -> a + b` — the annotation is the only practical reason to use it.

⚠️ INTERVIEW TIP

Mixing is illegal: `(var a, b)` won't compile. And you can't mix `var` with explicit types in the same lambda.

Q26 Explain the new String methods in Java 11.

Beginner | Java 11

💡 THEORY

Java 11 added: `isBlank()` (true if empty or only whitespace), `strip()/stripLeading()/stripTrailing()` (Unicode-aware trim), `repeat(n)`, and `lines()` (a stream of lines, split on line terminators).

⌨️ CODE

Strings11.java

```
"  ".isBlank();                // true
"\u2000hi\u2000".strip();      // "hi"
"=*".repeat(3);               // "=*=*=*"
"a\nb\nc".lines().count();    // 3
```

▶️ OUTPUT

`strip()` removes Unicode whitespace that the old `trim()` misses (it only handles chars ≤ U+0020).

⚠️ INTERVIEW TIP

The classic gotcha: `trim()` vs `strip()`. `trim()` is ASCII-only and predates Unicode; prefer `strip()` for modern text.

Q27 What's the new HttpClient and how is it better than HttpURLConnection?

Intermediate | Java 11

💡 THEORY

`java.net.http.HttpClient` (standard in 11) supports HTTP/2, WebSocket, sync and async (`CompletableFuture`) calls, and a fluent builder — replacing the clunky, blocking `HttpURLConnection`. It's reusable and thread-safe.

⌨️ CODE

HttpClient.java

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest req = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com"))
    .GET().build();
client.sendAsync(req, BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println);
```

▶️ OUTPUT

Non-blocking GET; the body prints when the response arrives, all on the client's executor.

🎯 SCENARIO

It first appeared as an *incubator* module in Java 9, became **standard** in Java 11 — interviewers test whether you know that timeline.

Q28 How do Files.readString / writeString simplify file I/O?

Beginner | Java 11

💡 THEORY

These convenience methods read/write an entire file to/from a `String` in one call, defaulting to UTF-8. They replace the old `Files.readAllBytes(...)` + `new String(bytes, UTF_8)` dance.

⌨️ CODE

FilesIO.java

```
Path p = Path.of("config.json");
String json = Files.readString(p);
Files.writeString(Path.of("out.txt"), "done");
```

▶️ OUTPUT

Whole file in/out as a `String`, UTF-8 by default — one line each.

⚠️ INTERVIEW TIP

For huge files this loads everything into memory — prefer `Files.lines()` (a lazy stream) for large or unbounded inputs.

Q29 What does Collectors.toUnmodifiableList do?

Beginner | Java 10

💡 THEORY

Introduced in Java 10, it collects a stream directly into an immutable list (and there are `toUnmodifiableSet/Map` variants). The result rejects all mutations, giving you defensive immutability without a wrapping call.

⌨️ CODE

Unmodifiable.java

```
List<String> out = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toUnmodifiableList());
out.add("X"); // UnsupportedOperationException
```

▶️ OUTPUT

Returns an immutable list; any structural change throws `UnsupportedOperationException`.

⚠️ INTERVIEW TIP

In Java 16+, `Stream.toList()` is shorter and also returns an unmodifiable list — but it permits `null` elements, whereas `toUnmodifiableList()` does not.

Q30 Can you run a Java source file without compiling it first?

Beginner | Java 11

💡 THEORY

Yes — Java 11's "Launch Single-File Source-Code Programs" (JEP 330) lets `java Hello.java` compile in memory and run immediately. Great for scripts and learning; no explicit `javac` step.

⌨️ CODE

terminal

```
$ java Hello.java
Hello, Java 11!

# shebang style script also works
#!/usr/bin/java --source 11
```

▶️ OUTPUT

The file is compiled to memory and executed in one step — no `.class` file is written.

⚠️ INTERVIEW TIP

Only works for a **single** source file. Multi-file projects still need normal compilation or a build tool.

Q31 Why did so many teams migrate from Java 8 straight to 11?

Intermediate | Java 11

💡 THEORY

Java 11 is an **LTS** (long-term support) release, and 8 → 11 captured a huge batch of improvements at once. But there were breaking changes: Java EE / CORBA modules (JAXB, JAX-WS) were removed, and the JRE was no longer shipped separately. Teams had to add those as explicit dependencies.

⌨️ CODE

pom.xml

```
<!-- JAXB removed from JDK 11, add it back -->
<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
</dependency>
```

▶️ OUTPUT

Code relying on `javax.xml.bind` fails to compile on 11 until the dependency is re-added explicitly.

🎯 SCENARIO

Common migration interview: *"What broke going 8→11?"* — removed Java EE modules, stricter encapsulation of internal JDK APIs (`sun.misc.Unsafe` warnings), and the single-JRE change.



JAVA 12 (2019) → JAVA 15 (2020)

Java 12–15

Modern Syntax Takes Shape

A rapid run of preview features that modernised the language: switch expressions, text blocks, records, pattern matching for `instanceof`, sealed classes, the teeing collector, and helpful NullPointerExceptions — most previewed here before standardising in 16/17.

Q32 What are switch expressions and how do they differ from switch statements?

Intermediate | Java 14

THEORY

Switch expressions (preview in 12/13, standard in 14) *return a value* and use arrow `→` labels with no fall-through. Multiple labels can share one arm, and `yield` returns a value from a block. They eliminate the classic missing-`break` bug.

CODE

SwitchExpr.java

```
int days = switch (month) {
  case JAN, MAR, MAY, JUL, AUG, OCT, DEC → 31;
  case APR, JUN, SEP, NOV → 30;
  case FEB → { yield leap ? 29 : 28; }
};
```

OUTPUT

Returns the day count directly — no `break`, no fall-through, and the compiler checks exhaustiveness for enums.

INTERVIEW TIP

Arrow labels don't fall through; colon labels (`case X:`) still do. `yield` returns from a switch *expression*; `break` / `return` belong to statements.

Q33 What are text blocks and what do they solve?

Beginner | Java 15

💡 THEORY

Text blocks (preview 13/14, standard 15) are multi-line string literals delimited by `"""`. They remove escaping noise for JSON, SQL, HTML and auto-manage indentation (incidental whitespace is stripped relative to the closing delimiter).

⌨️ CODE

TextBlock.java

```
String json = """
    {
      "name": "Ada",
      "role": "engineer"
    }""";
```

▶️ OUTPUT

A clean multi-line JSON string with no `\n` or escaped quotes — indentation is normalised to the closing `"""`.

⚠️ INTERVIEW TIP

Know the trailing-space escape `\s` and the line-continuation `\` (suppresses the newline). The opening `"""` must be followed by a line break.

Q34 What is a record (preview in Java 14)?

Intermediate | Java 14

💡 THEORY

A record is a transparent carrier for immutable data. The compiler generates the canonical constructor, private final fields, accessors, and value-based `equals` / `hashCode` / `toString`. It debuted as a preview in 14, refined in 15, and became standard in 16.

⌨️ CODE

Point.java

```
record Point(int x, int y) {
  // compact ctor for validation
  Point {
    if (x < 0 || y < 0)
      throw new IllegalArgumentException();
  }
}
```

▶️ OUTPUT

`new Point(1,2).toString()` → `Point[x=1, y=2]`; two points with equal fields are `equals`.

⚠️ INTERVIEW TIP

Records are implicitly `final`, can't extend a class (they extend `java.lang.Record`), but *can* implement interfaces. Components are shallowly immutable.

Q35 Explain pattern matching for instanceof.

🟡 Intermediate | Java 14

💡 THEORY

Pattern matching for `instanceof` (preview 14, standard 16) binds a variable when the type check succeeds, removing the redundant explicit cast. The binding's scope is flow-sensitive — available wherever the check is provably true.

⌨️ CODE

PatternInstanceof.java

```
// before
if (o instanceof String) {
  String s = (String) o;  System.out.println(s.length());
}
// after
if (o instanceof String s) {
  System.out.println(s.length());
}
```

▶️ OUTPUT

Same behaviour, no explicit cast. `s` is in scope only where `o` is guaranteed to be a `String`.

⚠️ INTERVIEW TIP

Flow scoping is clever: `if (!(o instanceof String s)) return; s.length();` works because past the guard `s` must be bound.

Q36 What are sealed classes (preview in Java 15)?

Advanced | Java 15

💡 THEORY

A sealed class/interface restricts which types may extend or implement it via a `permits` clause. This models closed hierarchies (algebraic data types) and enables exhaustive pattern matching. Previewed in 15/16, standard in 17.

⌨️ CODE

Sealed.java

```
sealed interface Shape
    permits Circle, Square {}

record Circle(double r) implements Shape {}
record Square(double s) implements Shape {}
```

▶️ OUTPUT

Only `Circle` and `Square` may implement `Shape`; any other implementer fails to compile.

⚠️ INTERVIEW TIP

Each permitted subtype must be `final`, `sealed`, or `non-sealed`. Combined with switch pattern matching, the compiler can prove exhaustiveness — no `default` needed.

Q37 What are Helpful NullPointerExceptions (Java 14)?

Beginner | Java 14

💡 THEORY

JEP 358 makes the JVM describe *exactly which* variable was null in a chained expression, instead of a bare line number. Enabled by default since Java 15.

⌨️ CODE

HelpfulNPE.java

```
order.getCustomer().getAddress().getCity();
// if getAddress() returns null:
```

▶️ OUTPUT

`Cannot invoke "Address.getCity()" because the return value of "Customer.getAddress()" is null` — pinpoints the exact culprit.

⚠️ INTERVIEW TIP

Pre-15 you can opt in with `-XX:+ShowCodeDetailsInExceptionMessages`. It's a runtime/diagnostic feature, not a language change.

Q38 What is the teeing collector (Java 12)?

Advanced | Java 12

💡 THEORY

`Collectors.teeing(c1, c2, merger)` feeds each element to **two** downstream collectors simultaneously, then merges their results with a bi-function. Perfect for computing two aggregates in a single pass.

⌨️ CODE

Teeing.java

```
double avg = nums.stream().collect(
  Collectors.teeing(
    Collectors.summingDouble(n → n),
    Collectors.counting(),
    (sum, cnt) → sum / cnt));
```

▶️ OUTPUT

Computes the average in one stream traversal by combining a sum and a count.

🎯 SCENARIO

When you need min *and* max, or sum *and* count, without iterating twice — `teeing` is the clean answer interviewers look for.

Q39 What does the term "preview feature" mean in Java?

🟡 Intermediate | Java 12+

💡 THEORY

A preview feature is fully implemented but not yet final — it ships for feedback and may change or be removed. You must opt in at both compile and run time with `--enable-preview` and a target release. This lets the JDK iterate (e.g. records, sealed classes, pattern matching) before committing.

⌨️ CODE

terminal

```
$ javac --release 21 --enable-preview App.java
$ java  --enable-preview App
```

▶️ OUTPUT

Without `--enable-preview`, code using a preview feature won't compile or run on that release.

⚠️ INTERVIEW TIP

Preview classfiles are version-locked: a class compiled with preview on JDK 21 won't run on JDK 22. Never ship preview features to production.

Q40 How do compact constructors in records work?

🟡 Intermediate | Java 14

💡 THEORY

A compact constructor omits the parameter list and the trailing field assignments. You write only validation/normalization; the compiler appends `this.x = x;` for every component automatically. You may reassign the parameters to normalize values before they're stored.

⌨️ CODE

Range.java

```
record Range(int lo, int hi) {
  Range {                  // compact ctor
    if (lo > hi) { int t = lo; lo = hi; hi = t; }
  }                        // this.lo/this.hi auto-assigned
}
```

▶️ OUTPUT

`new Range(5, 1)` stores `lo=1, hi=5` — normalized, with no explicit field writes.

⚠️ INTERVIEW TIP

You cannot assign `this.lo` inside a compact constructor — only the parameter. Assigning fields directly is the job of a full canonical constructor.

Q41 Why was the G1 garbage collector made the default, and what is ZGC?

Advanced | Java 12–15

💡 THEORY

G1 (default since Java 9) balances throughput and predictable pause times by collecting regions incrementally. ZGC (experimental in 11/15) is a low-latency collector targeting sub-millisecond pauses regardless of heap size, using colored pointers and concurrent relocation. Shenandoah is a similar concurrent collector.

⌨️ CODE

terminal

```
$ java -XX:+UseZGC -Xmx16g MyApp
$ java -XX:+UseG1GC MyApp        # default
```

▶️ OUTPUT

ZGC keeps pause times nearly constant even as the heap grows into hundreds of GB — pauses don't scale with heap size.

🎯 SCENARIO

Low-latency trading or real-time systems pick ZGC/Shenandoah; batch/throughput jobs often stick with G1 or Parallel GC. Java 21 added **generational ZGC** for better efficiency.



JAVA 16 (2021) · JAVA 17 LTS (SEP 2021)

Java 16–17

The Features Go Standard

Java 17 — the third major LTS — graduated years of previews to **standard**: records, pattern matching for `instanceof`, and sealed classes all became final, joined by `Stream.toList()`, an enhanced pseudo-random generator API, and switch pattern matching (still preview).

Java 16–17 · Standardized

Q42 Records became standard in Java 16 — recap the full feature.

Intermediate | Java 16

💡 THEORY

As of Java 16, records are a permanent language feature. They model immutable data with minimal ceremony, support compact constructors, static factories, additional methods, and implementing interfaces — but no inheritance and no mutable state.

⌨️ CODE

Money.java

```
record Money(long cents, String ccy) {
  static Money usd(long c) { return new Money(c, "USD"); }
  String pretty() { return ccy + " " + cents/100.0; }
}
```

▶️ OUTPUT

`Money.usd(1050).pretty()` → `USD 10.5`. Auto `equals` / `hashCode` make it a perfect map key.

🎯 SCENARIO

Ideal for DTOs, value objects, and as keys in maps/sets. Follow-up: *"Can a record have static fields?"* — yes; only *instance* fields are restricted to the components.

Q43 How is Stream.toList() different from collect(toList())?

Intermediate | Java 16

💡 THEORY

`Stream.toList()` (Java 16) is a concise terminal op returning an **unmodifiable** list. `collect(Collectors.toList())` returns a list with **no** guarantee on mutability or type (usually a mutable `ArrayList`).

⌨️ CODE

ToList.java

```
List<Integer> a = s.stream().map(x→x*2).toList();
a.add(9); // UnsupportedOperationException
```

▶️ OUTPUT

`toList()` yields an immutable list; mutating it throws. Shorter and clearer than the collector form.

⚠️ INTERVIEW TIP

`toList()` allows `null` elements; `toUnmodifiableList()` does not. If you migrated and started getting `UnsupportedOperationException`, that's the cause.

Q44 Sealed classes are standard in 17 — how do they enable exhaustive switches?

Advanced | Java 17

💡 THEORY

Because a sealed type lists all permitted subtypes, the compiler knows the full set. Combined with pattern matching for switch, it can verify every case is handled — so no `default` branch is needed, and adding a new subtype causes a compile error until you handle it.

⌨️ CODE

SealedSwitch.java

```
sealed interface Shape permits Circle, Square {}
double area = switch (shape) {
  case Circle c → Math.PI * c.r() * c.r();
  case Square s → s.side() * s.side();
}; // no default needed — exhaustive
```

▶️ OUTPUT

Compiles without a `default`. Add a `Triangle` to `permits` and this switch fails until you handle it.

⚠️ INTERVIEW TIP

This is the "make illegal states unrepresentable" pattern — closed hierarchies + exhaustive switches catch missing cases at compile time, not in production.

Q45 What is the enhanced pseudo-random number generator API (Java 17)?

Intermediate | Java 17

💡 THEORY

JEP 356 adds `RandomGenerator` interfaces and a `RandomGeneratorFactory`, unifying old and new algorithms (e.g. LXM, Xoshiro). You can select an algorithm by name and stream random values — far more flexible than the legacy `java.util.Random`.

⌨️ CODE

PRNG.java

```
RandomGenerator rng =
    RandomGeneratorFactory.of("L64X128MixRandom")
        .create(42);
rng.ints(5, 0, 100).forEach(System.out::println);
```

▶️ OUTPUT

Five reproducible ints in [0,100). The algorithm is pluggable and seedable for deterministic tests.

⚠️ INTERVIEW TIP

`java.util.Random` now implements `RandomGenerator`, so old code keeps working. For security, still use `SecureRandom`.

Q46 What is pattern matching for switch (preview in Java 17)?

Advanced | Java 17

💡 THEORY

Previewed in Java 17 (JEP 406), it lets `case` labels test types and bind variables, with optional guard clauses (`when`). It became standard in Java 21. It replaces long `if-else` `instanceof` chains with a single, exhaustive switch.

⌨️ CODE

SwitchPattern.java

```
String describe = switch (obj) {
  case Integer i when i > 0 → "positive int";
  case Integer i            → "non-positive int";
  case String s             → "string of len " + s.length();
  case null                 → "null!";
  default                   → "other";
};
```

▶️ OUTPUT

Type-tests, binds, and applies a guard in one construct. Note switch can now handle `null` explicitly.

⚠️ INTERVIEW TIP

Order matters: a guarded `case` must precede its unguarded sibling, and dominance rules mean a broader pattern can't shadow a narrower one. The compiler enforces this.

Q47 Why was Java 17 such an important LTS for the ecosystem?

Beginner | Java 17

💡 THEORY

Java 17 consolidated three years of features into a stable LTS and became the new baseline: Spring Boot 3 requires Java 17+, and most frameworks made it the minimum. It also finalized the removal/encapsulation of internal APIs and applied strong encapsulation of JDK internals by default.

⌨️ CODE

pom.xml

```
<properties>
  <java.version>17</java.version>
</properties>
<!— Spring Boot 3.x baseline —>
```

▶️ OUTPUT

Adopting Spring Boot 3 / Jakarta EE 9+ forces the Java 17 baseline — a major migration driver.

🎯 SCENARIO

"Why upgrade from 11 to 17?" — LTS support window, framework requirements, records/sealed/pattern matching going standard, and meaningful GC and startup improvements.

Q48 What does strong encapsulation of JDK internals mean (JEP 403)?

Advanced | Java 16/17

💡 THEORY

From Java 16 (JEP 396) and finalized in 17 (JEP 403), internal packages like `sun.*` are strongly encapsulated by default. Reflective access that used to emit a warning now throws `InaccessibleObjectException` unless explicitly opened with `--add-opens`.

⌨️ CODE

terminal

```
$ java --add-opens java.base/java.lang=ALL-UNNAMED App
# needed by some old libraries doing deep reflection
```

▶️ OUTPUT

Without the flag, frameworks doing deep reflection on JDK internals fail at runtime — a common upgrade blocker.

⚠️ INTERVIEW TIP

This is why older versions of libraries (e.g. some Hibernate/Lombok builds) needed `--add-opens` on Java 16/17. The fix is to upgrade the library, not rely on the flag forever.

Q49 Can records implement interfaces and have extra methods?

Beginner | Java 17

💡 THEORY

Yes. A record cannot extend a class, but it can implement any number of interfaces and declare additional instance methods, static methods, and static fields. This makes records first-class participants in sealed hierarchies and pattern matching.

⌨️ CODE

Named.java

```
interface Named { String name(); }

record User(String name, int age) implements Named {
  boolean isAdult() { return age ⩾ 18; }
}
```

▶️ OUTPUT

The record's auto-generated `name()` accessor satisfies the interface; `isAdult()` adds derived behavior.

⚠️ INTERVIEW TIP

You may override an accessor or add a canonical constructor, but the record's core contract (immutability, value equality) must hold or you've defeated its purpose.



JAVA 18 (2022) → JAVA 21 LTS (SEP 2023)

Java 18–21

The Project Loom Era

Java 21 — the current LTS — is a landmark: **virtual threads** and **pattern matching for switch** went standard, alongside record patterns, sequenced collections, and generational ZGC. Structured concurrency, scoped values, and string templates arrived as previews.

Q50 What are virtual threads (Project Loom, standard in Java 21)?

Advanced · Java 21

THEORY

Virtual threads are lightweight threads managed by the JVM, not the OS. Millions can run concurrently because they're cheap (a few hundred bytes) and the JVM *unmounts* them from a carrier (platform) thread whenever they block on I/O. This brings simple, blocking-style code to massive concurrency — no async/reactive complexity.

CODE

VirtualThreads.java

```
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
  IntStream.range(0, 1_000_000).forEach(i →
      exec.submit(() → { Thread.sleep(Duration.ofSeconds(1)); return i;
}));
}
// also: Thread.ofVirtual().start(runnable);
```

OUTPUT

A million concurrent tasks run on a handful of OS threads — something impossible with platform threads (which would exhaust memory).

SCENARIO

Ideal for thread-per-request servers doing blocking I/O. Follow-up: *"What is pinning?"* — a virtual thread stuck to its carrier inside a `synchronized` block or native call; prefer `ReentrantLock` to avoid it.

Q51 Virtual threads vs platform threads — the key differences?

Advanced | Java 21

💡 THEORY

A platform thread is a thin wrapper over an OS thread — expensive (~1MB stack), limited to thousands. A virtual thread is scheduled by the JVM onto a small pool of carrier threads, costs almost nothing, and is meant to be short-lived and never pooled. You scale by creating more, not by pooling.

⌨️ CODE

ThreadKinds.java

```
Thread p = Thread.ofPlatform().start(task);   // OS thread
Thread v = Thread.ofVirtual().start(task);    // JVM thread
v.isVirtual(); // true
```

▶️ OUTPUT

Both run the same `Runnable`, but the virtual thread is multiplexed onto a shared carrier pool.

⚠️ INTERVIEW TIP

Never pool virtual threads and don't use them for CPU-bound work — they shine for blocking I/O. Thread-local heavy code should be reviewed; prefer scoped values.

Q52 What are record patterns (standard in Java 21)?

Advanced | Java 21

💡 THEORY

Record patterns (JEP 440) let you **deconstruct** a record directly in a pattern, binding its components — and they nest. Combined with switch, you destructure complex data in one expression.

⌨️ CODE

RecordPattern.java

```
record Point(int x, int y) {}
record Line(Point a, Point b) {}

String s = switch (obj) {
  case Line(Point(var x1, var y1), Point(var x2, var y2))
      → "("+x1+","+y1+")→("+x2+","+y2+")";
  default → "?";
};
```

▶️ OUTPUT

Nested deconstruction binds all four coordinates at once — no manual `.a().x()` chains.

⚠️ INTERVIEW TIP

Record patterns + sealed types = exhaustive, type-safe destructuring — Java's answer to functional-language pattern matching.

Q53 What are Sequenced Collections (Java 21)?

Intermediate | Java 21

💡 THEORY

JEP 431 adds `SequencedCollection`, `SequencedSet`, and `SequencedMap` — interfaces giving a uniform API for collections with a defined encounter order: `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, and `reversed()`.

⌨️ CODE

Sequenced.java

```
List<String> list = new ArrayList<>(List.of("a","b","c"));
list.getFirst();          // "a"
list.getLast();           // "c"
list.reversed();          // [c, b, a] (view)
```

▶️ OUTPUT

No more `list.get(list.size()-1)` — a single, consistent vocabulary across `List`, `Deque`, `LinkedHashSet`, etc.

⚠️ INTERVIEW TIP

`reversed()` returns a **view**, not a copy — changes write through. And `getFirst()` on an empty collection throws `NoSuchElementException`.

Q54 What is Structured Concurrency (preview in Java 21)?

Advanced | Java 21

💡 THEORY

Structured concurrency (JEP 453, preview) treats a group of related concurrent tasks as a single unit of work via `StructuredTaskScope`. If one subtask fails, siblings are cancelled; the scope ensures all complete before the parent proceeds — eliminating thread leaks and orphaned tasks.

⌨️ CODE

StructuredConcurrency.java

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
  var user  = scope.fork(() → fetchUser(id));
  var order = scope.fork(() → fetchOrder(id));
  scope.join().throwIfFailed();
  return new Result(user.get(), order.get());
}
```

▶️ OUTPUT

Both calls run concurrently on virtual threads; if either fails, the other is cancelled and the error propagates cleanly.

🎯 SCENARIO

Fan-out to several services and join results — like `CompletableFuture.allOf` but with proper cancellation, error handling, and observability tied to the call stack.

Q55 What are Scoped Values (preview in Java 21)?

Advanced | Java 21

💡 THEORY

Scoped values (JEP 446) are an immutable, structured alternative to `ThreadLocal`, designed for virtual threads. A value is bound for the dynamic extent of a `run` call and automatically inherited by forked tasks — no leaks, no manual cleanup.

⌨️ CODE

ScopedValue.java

```
final static ScopedValue<User> CURRENT = ScopedValue.newInstance();

ScopedValue.where(CURRENT, user)
    .run(() -> handleRequest());  // CURRENT.get() inside
```

▶️ OUTPUT

`CURRENT.get()` returns the bound user anywhere in the call tree, then is unbound when `run` returns.

⚠️ INTERVIEW TIP

Unlike `ThreadLocal`, scoped values are immutable and bounded — ideal when you spawn millions of virtual threads where per-thread mutable state would be wasteful and leak-prone.

Q56 What are String Templates (preview in Java 21)?

🟡 Intermediate | Java 21

💡 THEORY

String templates (JEP 430, preview) introduce template processors like `STR` that interpolate embedded expressions with `\{...}`. Unlike naive concatenation, custom processors (e.g. for SQL/HTML) can validate and escape inputs, preventing injection.

⌨️ CODE

StringTemplate.java

```
String name = "Ada"; int n = 42;
String msg = STR."Hello \{name}, you have \{n} messages";
```

▶️ OUTPUT

`Hello Ada, you have 42 messages` — interpolation is built into the language, not string glue.

⚠️ INTERVIEW TIP

This is a **preview** in 21 and was notably redesigned afterward — say "preview, syntax may change," not "available." Honesty about maturity scores points.

Q57 What is Generational ZGC (Java 21)?

Advanced | Java 21

💡 THEORY

JEP 439 adds generational support to ZGC, maintaining separate young and old generations. Since most objects die young, collecting the young generation more frequently improves efficiency — lower CPU and memory overhead while keeping ZGC's sub-millisecond pauses.

⌨️ CODE

terminal

```
$ java -XX:+UseZGC -XX:+ZGenerational MyApp
# generational mode became default for ZGC in Java 23
```

▶️ OUTPUT

Higher throughput and lower heap headroom than single-generation ZGC, with the same ultra-low pause guarantees.

🎯 SCENARIO

Large-heap, latency-sensitive services (caches, streaming, trading) benefit most. In Java 21 you opt in with `-XX:+ZGenerational`.

Q58 What is the Simple Web Server (Java 18)?

Beginner | Java 18

💡 THEORY

JEP 408 ships a minimal static-file HTTP server via the `jwebserver` command and a `SimpleFileServer` API. It's for prototyping, testing, and education — not production — giving a zero-dependency way to serve files.

⌨️ CODE

terminal

```
$ jwebserver -p 8000 -d /var/www
Serving /var/www on http://127.0.0.1:8000/ ...
```

▶️ OUTPUT

Instantly serves a directory over HTTP — handy for quick demos and front-end testing, no framework required.

⚠️ INTERVIEW TIP

It only handles `GET` and `HEAD` and has no auth — explicitly not meant for production traffic.

Q59 What is UTF-8 by default (Java 18, JEP 400)?

🟡 Intermediate | Java 18

💡 THEORY

Before Java 18, the default charset for file I/O depended on the OS/locale, causing "works on my machine" bugs. JEP 400 makes `UTF-8` the default charset consistently across all platforms for the standard APIs that derive a default charset.

⌨️ CODE

Charset.java

```
System.out.println(Charset.defaultCharset());
// Java 18+ : UTF-8 on every OS
new FileReader("data.txt"); // now UTF-8, not platform default
```

▶️ OUTPUT

`UTF-8` everywhere — the same bytes decode identically on Windows, Linux, and macOS.

⚠️ INTERVIEW TIP

If legacy code silently relied on a platform charset (e.g. Windows-1252), upgrading to 18 can change behaviour — set `-Dfile.encoding` to verify before/after.

Q60 How do virtual threads change typical Spring/web app design?

Advanced | Java 21

💡 THEORY

With virtual threads, the simple "thread-per-request" blocking model scales to huge concurrency without reactive complexity. Spring Boot 3.2+ enables them via one property; each request runs on its own virtual thread, so blocking JDBC/HTTP calls no longer starve a small platform-thread pool.

⌨️ CODE

application.properties

```
spring.threads.virtual.enabled=true
# Tomcat now serves each request on a virtual thread
```

▶️ OUTPUT

Throughput under heavy blocking I/O rises sharply with no code rewrite — you keep readable, sequential controllers.

🎯 SCENARIO

"Do virtual threads replace reactive (WebFlux)?" — For most blocking-I/O apps, yes, with far simpler code. Reactive still wins for streaming/back-pressure-heavy pipelines.

Q61 Summarize the LTS timeline and why it matters for upgrades.

Beginner | Java 8→21

💡 THEORY

LTS releases get years of support and are the safe production targets: **Java 8** (2014), **11** (2018), **17** (2021), **21** (2023), with **25** next (2025). Non-LTS releases ship every 6 months and are stepping stones for previews. Most enterprises hop LTS-to-LTS: 8 → 11 → 17 → 21.

⌨️ CODE

```
timeline

8  → 11  → 17  → 21   (LTS, ~every 2 yrs since 17)
9,10,12..16,18,19,20  (6-month feature releases)
```

▶️ OUTPUT

Targeting LTS keeps you on supported, patched runtimes while still receiving the features that matured across the interim releases.

⚠️ INTERVIEW TIP

Be precise about **preview vs standard** per version — interviewers specifically test whether you know, e.g., virtual threads were preview in 19/20 and standard in **21**.



You're Ready.

61 questions · Java 8 → 21 · covered end-to-end

Lambdas & Streams · Optional · Modules · var · HttpClient · Records · Sealed Classes · Pattern Matching · Virtual Threads · Structured Concurrency · Sequenced Collections

TechTribers

Keep building. Keep shipping. **Good luck** in the interview. 🚀